Native hashed-ecdl-tensor measurements on small prime-order curves

Study: `research/btn_ecdl_prime_20261029/` (PR #2888). Tool: `native/` (`btn-prime-scaling`, Rust), depending on the unmodified `crates/hashed-ecdl-tensor` crate (#2878). All numbers below are read from `native_runs/report.json`, produced by `btn-prime-scaling report native_runs` from the raw records `sizes.jsonl`, `counts.jsonl`, `solves.jsonl` and `eccp131_size.json`.

Unit. Step counts are the crate's `Meter` charges, deterministic diagnostics and not calibrated time units. **No wall-clock time is reported**, so no timing isolation applies. Host: a 4-core Claude Code cloud container (Linux).

Exact counts against brute force

16 contractions of the decomposition network (two slots, full and hash-filtered factor bases) at p = 5, 7, 11, 13. All match brute force: **`true`**. Backend `Functional` in every count: **`true`**. Visited assignments equal 2^{4d} in every count: **`true`**. Recursive (OpenAI Appendix D) backend used anywhere: **`false`**.

p	d	brute	backend	assignments	steps	steps / assignment
5	3	match	Some(Functional)	4096	4485600	1095
5	3	match	Some(Functional)	4096	4485600	1095
5	3	match	Some(Functional)	4096	3523331	860
5	3	match	Some(Functional)	4096	3544187	865
7	3	match	Some(Functional)	4096	9627056	2350
7	3	match	Some(Functional)	4096	9627056	2350
7	3	match	Some(Functional)	4096	6463893	1578
7	3	match	Some(Functional)	4096	6463893	1578
11	4	match	Some(Functional)	65536	69188532	1056
11	4	match	Some(Functional)	65536	69188532	1056
11	4	match	Some(Functional)	65536	58303887	890
11	4	match	Some(Functional)	65536	58318774	890
13	4	match	Some(Functional)	65536	107725686	1644
13	4	match	Some(Functional)	65536	107725686	1644
13	4	match	Some(Functional)	65536	93262386	1423
13	4	match	Some(Functional)	65536	93277273	1423

End-to-end solves (`reduction::solve`, seeded)

p	d	order r	seed	secret	recovered	relations	steps
5	3	7	1	2	2 (verified)	13	210937575
5	3	7	2	1	1 (verified)	13	211864590
7	3	13	1	3	3 (verified)	25	738643764

Network size

d	p	edges	vertices	max degree
3	5	31624	8402	132
4	11	56480	12860	132
5	17	57952	13722	132
6	37	59580	14662	132
7	67	61120	15558	132
8	131	64848	16552	132
9	257	114192	19438	132
10	521	115816	20376	132

11	1031	117440	21314	132
12	2053	118916	22178	132
13	4099	120456	23074	132
14	8209	122008	23976	132
15	16411	123648	25182	260
16	32771	127920	31176	260
17	65537	223736	35474	260

ECCp-131 network size: **not recorded** — the `size-json` build did not finish, stuck in AKS primality certification of the 131-bit prime (see "Setup bottleneck"), not in network construction.

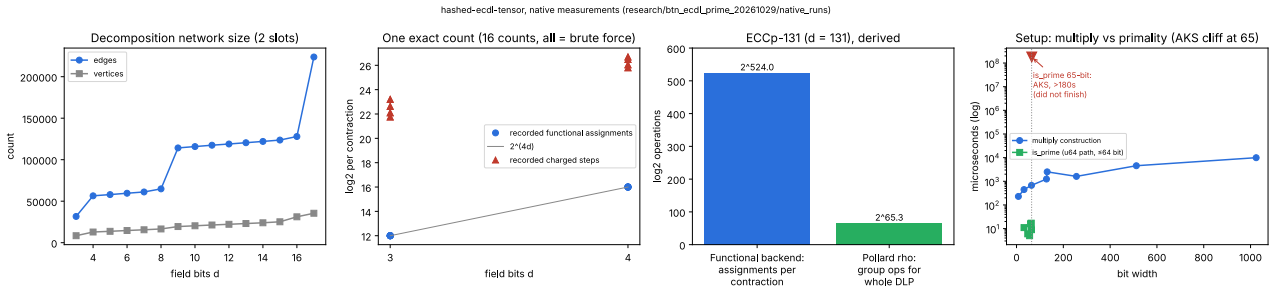


Figure 1: Recorded sizes, per-count assignments and steps, and the derived ECCp-131 comparison. Source: `native_runs/report.json`.

Setup bottleneck: primality certification above 64 bits

Before the contraction cost matters, there is a harder practical wall: `is_prime` (used by `Curve::new` and `FactorBase::new` to certify the field and subgroup primes) dispatches to a fast u64 path at <= 64 bits and to **AKS** above.

At <= 64 bits every certification took at most 17 µs. At 65 bits (AKS) it did **not finish in 180 s**, a jump of more than seven orders of magnitude at a single bit. ECCp-131's prime is 131-bit, so the curve cannot even be constructed in reasonable time; the 131-bit `size-json` build was stuck here, not in the network.

By contrast, **network construction is fast at every width**: the Fourier integer-multiply builds in 8-bit: 229 µs, 32-bit: 447 µs, 64-bit: 674 µs, 128-bit: 1244 µs, 131-bit: 2508 µs, 256-bit: 1610 µs, 512-bit: 4550 µs, 1024-bit: 9950 µs. So the blockers, in order, are (1) prime certification above 64 bits (AKS), then (2) the recursion crossover below. Neither is close to feasible.

The recursion crossover (headline)

The only part of the algorithm that could beat exponential cost is the recursive backend (OpenAI Appendix D.4). In the crate's `Counter::count`, it is selected only when root_u $= \lambda/131072$ is at least 1, with $\lambda = \lceil \log_2(2 \cdot \text{edges}) \rceil$. That needs $\lambda \geq 131072$, i.e. at least 2^{131070} edges (further gates only raise it).

The largest network built here is $2^{17.8}$ edges. The gap to the crossover is about 2^{131052}. Across all 16 counts and 3 solves the recursive backend was selected `false` times: every exact count used the fallback (`Functional/Direct`), whose cost is $2^{\text{free input bits}}$ per contraction.

This threshold is faithful to the manuscript, which states a saving only above networks of size 2^17 = 131072 (manuscript's 2^(2^17)). No prime-field curve produces a network near it, so this implementation gives no sub-exponential behaviour at any reachable size, including ECCp-131.

Derived: ECCp-131

The backend actually used at every measured size is `Functional`, which visits $2^{\text{free input bits}}$ assignments. With two slots at d = 131 that is 2^{524} assignments per contraction (Functional backend visits 2^(free input bits); measured equal to 2^(4d) at d=3,4; two slots at d=131 give 4*131 = 524 free bits). Pollard rho on ECCp-131 needs about $2^{65.33}$ group operations for the whole discrete logarithm. The recursive backend did not run at any measured size; this report says nothing about its cost.

What this does not show

No ECCp-131 scalar is computed or attempted. No speed-up is claimed. The recursive backend's asymptotic bound (Bochum v0.2, conditional on OpenAI's unreviewed Theorem D.1) is not tested. No row is added to `benchmarks/README.md`: no wall-clock number is reported, and the validation gate admits no row without an independent replay certificate, validator report and matched rho reference.